EPIM: Efficient Processing-In-Memory Accelerators based on Epitome

Chenyu Wang[1*],Zhen Dong[2*†],Daquan Zhou[3*†],Zhenhua Zhu[1],Yu Wang[1†],Jiashi Feng[3],Kurt Keutzer[2]
[1]Tsinghua University [2]UC Berkeley [3]ByteDance
[*] Equal contribution [†] Corresponding to: zhendong@berkeley.edu, zhoudaquan21@gmail.com, yu-wang@mail.tsinghua.edu.cn

ABSTRACT

The utilization of large-scale neural networks on Processing-In-Memory (PIM) accelerators encounters challenges due to constrained on-chip memory capacity. To tackle this issue, current works explore model compression algorithms to reduce the size of Convolutional Neural Networks (CNNs). Most of these algorithms either aim to represent neural operators with reduced-size parameters (e.g., quantization) or search for the best combinations of neural operators (e.g., neural architecture search). Designing neural operators to align with PIM accelerators' specifications is an area that warrants further study. In this paper, we introduce the Epitome, a lightweight neural operator offering convolution-like functionality, to craft memory-efficient CNN operators for PIM accelerators (EPIM). On the software side, we evaluate epitomes' latency and energy on PIM accelerators and introduce a PIM-aware layer-wise design method to enhance their hardware efficiency. We apply epitome-aware quantization to further reduce the size of epitomes. On the hardware side, we modify the datapath of current PIM accelerators to accommodate epitomes and implement a feature map reuse technique to reduce computation cost. Experimental results reveal that our 3-bit quantized EPIM-ResNet50 attains 71.59% top-1 accuracy on ImageNet, reducing crossbar areas by 30.65×. EPIM surpasses the state-of-the-art pruning methods on PIM.

1 INTRODUCTION

The emergence of Processing-In-Memory (PIM) accelerators offers a promising approach to boost the computational efficiency of neural network operations [1]. Leveraging the capability to conduct computations directly within memory and eliminating the need for frequent data transfers between memory and computing units, PIM accelerators offer a substantial improvement in energy efficiency. Remarkably, these accelerators surpass the performance of GPU and CMOS ASIC solutions by more than 100-fold [1, 7].

Despite the advances in current PIM accelerators, they still face certain challenges. One challenge arises from the higher cost of writing to emerging memory compared to reading from it. Due to this limitation, PIM accelerators typically require loading all neural network weights onto memristor crossbars prior to conducting computations. However, modern large-scale neural network models have seen a significant increase in parameter sizes. Even for the existing PIM accelerators with maximum memristors, there still exists a significant challenge in deploying a standard backbone model like ResNet-50[10]. As such, the deployment of advanced Convolutional Neural Networks (CNN) models on PIM is hindered.

To address this challenge, researchers have proposed methods that fall into two main categories. Various methods, including pruning and quantization, aim to represent neural operator weights using a reduced number of parameters[2, 9]. Conversely, approaches such as neural architecture search concentrates on identifying optimal combinations of neural operators[6, 8]. Despite the advancements, the majority of these studies have focused on adapting existing convolution operators. The crucial endeavor of developing compact and high-performance operators specifically tailored for PIM accelerators has largely been overlooked.

Designing an operator for PIM requires meeting essential standards. For an effective PIM-specific neural operator, it must address memristor limitations by using fewer memristors than current convolution operators without sacrificing accuracy. Additionally, the operator should require minimal adjustments to existing software and PIM accelerators, ensuring an efficient and seamless transition to the new design.

To craft such neural operator, we began by reviewing existing literature. Prior research highlights the potential of leveraging overlaps in convolution kernels, introducing a compact neural operator termed "epitome"[11]. This operator reconstructs convolution kernels from a smaller epitome tensor. A more compact neural network can be crafted by replacing the its convolutions with epitomes. We identified that, with software and hardware adjustments, epitome could be adapted to PIM accelerators. Specifically, from the hardware side, epitome only introduces minor modifications to the data path. Hence, we adopted epitome as our foundation. By integrating epitome with our proposed methods, we achieved notable compression rates while maintaining a high accuracy.

Our contributions can be summarized as follows:

- We pioneer the utilization of epitome as an efficient neural operator for PIM accelerators, which is a distinct perspective compared to existing model compression approaches.
- We recognize the correlation between epitome shape and both latency and energy consumption. Furthermore, we implement a PIM-aware layer-wise design method to optimize the epitome shape.
- We design three modules in the data path—IFRT, IFAT, and OFAT—to facilitate the deployment of epitome operators. We design a feature map reuse technique termed as "Channel Wrapping" to further reduce computation cost.
- We propose an epitome-aware quantization method. This approach improves accuracy for epitomes under ultra-low bit quantization.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

ACM ISBN 979-8-4007-0601-1/24/06.
https://doi.org/10.1145/3649329.3657377

Our proposed methods yield outstanding results. Our 3-bit quantized EPIM-ResNet50 model demonstrates remarkable performance by achieving an accuracy of 71.59% on the challenging ImageNet dataset with a crossbar compression rate of 30.65×.

2 RELATED WORK

2.1 Process-In-Memory Architecture for CNN Acceleration

Commonly, PIM architecture consists of multiple memristor crossbars. The weight matrices of CNN are mapped as the conductance of the memristor crossbar cells. However, the writing latency of the memristor crossbar cell is multiple times larger than the reading latency[1]. Before inference, the weight matrices of all CNN layers need to be mapped onto memristor crossbars. Such restrictions stimulate the need to support model compression algorithms on PIM architecture.

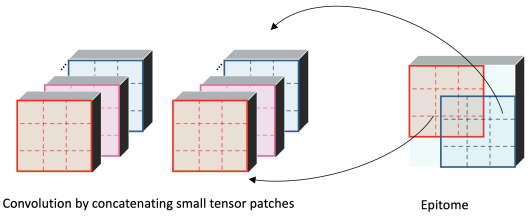


Figure 1: Epitome's Reconstruction into Convolution

2.2 Epitome

The compact neural operator, epitome, represents the neural network with a compact set of parameters $E = \{E_i\}$, and a sampler τ. $\{E_i\}$ (epitomes) are a set of learnable parameters whose size are significantly smaller than the convolutions', and the sampler τ repeatedly samples small patches from the epitomes. Each time of sampling, a sampler returns a small patch of epitome:

$$E_s = E_i[p : p + w, q : q + h, c_{in} : c_{in} + \beta_1, c_{out} : c_{out} + \beta_2], \quad (1)$$

where (p, q, c_{in}, c_{out}) denote the starting index of the sub-tensor in the epitome and (w, h, β_1, β_2) denotes the length of the sub-tensor along each dimension. The sampling process is repeated until the fetched small pieces of weight tensors can be concatenated to match the dimensions of the convolution. Figure 1 is a visual illustration of such process. Unlike convolutions, whose shapes are determined by the input and output channels, epitomes have greater flexibility in shape. They can be of four-dimensional tensors with any size, as long as their sampled patches can reconstruct the convolutions[11].

2.3 Quantization

Quantization methods reduce model size by using low bitwidth for weights and activations in neural networks[4]. In quantization, the quantizer maps weights and activations into integers with a real-valued scaling factor S and an integer zero point Z as follows:

$$Q(r) = \text{Int}(r/S) - Z \quad (2)$$

where Q is the quantization operator, r is a real-valued input, and Int() represents a rounding operation. For a k-bit quantization, the scaling factor can be determined as follows:

$$S = \frac{\beta - \alpha}{2^k - 1} \quad (3)$$

$[\alpha, \beta]$ denotes the quantization range. A straightforward choice is to use the min/max of the signal for the clipping range, i.e., $\alpha = r_{min}$, and $\beta = r_{max}$.

3 FRAMEWORK OVERVIEW

The general framework for designing and training neural networks with epitomes is presented in Figure 2(a). EPIM begins with any convolution-based neural network. Subsequently, epitome designer is used to replace the convolutions by epitomes. This task can be performed manually or by using an evolutionary search to design a well-performing set of epitomes, informed by feedback from a simulator. Post-design, we attain a neural network where the operators are epitomes. After training, the epitome designer converts the floating point model to fixed-point. Then, we modify the data path and design the feature map reuse strategy, termed as "Channel Wrapping". After these steps, a well-crafted epitome based neural network can be deployed on PIM accelerators.

4 EPIM DESIGN METHOD

This section introduces our method EPIM. Section 4.1 shows the adjustments of epitome on PIM accelerators. Section 4.2 presents details of our quantization method. Section 4.3 describes the specifications of our PIM accelerator.

4.1 Mapping and Adjusting Epitomes for PIM Accelerators

Just like convolutions, epitomes are four-dimensional tensors that can be mapped onto memristors in a similar manner. The distinction between epitomes and convolutions arises during inference: while all word lines and bit lines corresponding to a convolution's weight are activated all together, an epitome causes a memristor crossbar to be activated multiple times. Each activation only engages the word lines and bit lines associated with small patches sampled from the epitome. In our implementation, we use the same mapping strategy as [13].

Motivated by the size flexibility of the epitomes, we can adjust their shapes to better utilize memristors. Specifically, we aim for c_{out} and $c_{in} \times p \times q$ to align as integral multiples of the crossbar size. Following the mapping approach from [13], where c_{in}, p and q dimensions correspond to word lines, while c_{out} dimension corresponds to bit lines on memristor crossbars. By setting to these dimensions to desired values, we ensure epitomes make full use of memristor crossbars.

4.2 Quantization Method for Epitome

When directly quantizing the epitome with ultra-low precision, we observed a notable decrease in accuracy. Table 2 indicates that while the 5-bit quantized weights for ResNet-50's epitome achieve 73.59% accuracy, this drops to 69.95% for 3-bit weights. To enhance accuracy under such low-bit quantization, we suggest two adjustments:

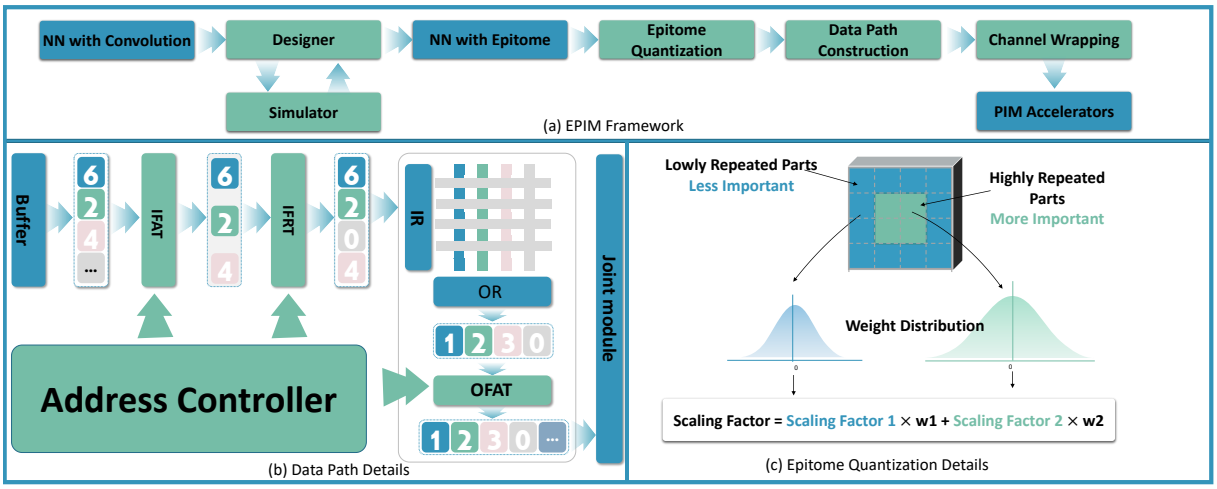


Figure 2: Illustration of EPIM Framework.

Firstly, given the parallel computation between PIM accelerator crossbars, we allocate a scaling factor to each crossbar. Secondly, we observe that the epitome reconstructs convolution by sampling overlapping small patches, with certain patches from the epitome being repeated more often than others in the reconstructed convolution. When quantizing the epitome, it's logical to assign different importance levels to different parts based on their repetition frequency. As depicted in Figure 2(c), we notice that the center parts (green) of epitomes are usually repeated more frequently than the other parts (blue). We calculate the weighted sum of the epitome's maximum and minimum values in the center parts and other parts as quantization range α and β, based on these repetitions. The detailed calculation process is as follows:

$$\alpha = w_1 \times (r_{min})_{overlap} + w_2 \times (r_{min})_{others} \quad (4)$$

$$\beta = w_1 \times (r_{max})_{overlap} + w_2 \times (r_{max})_{others} \quad (5)$$

w_1 and w_2 are two hyperparameters denoting the significance of distinct sections of the epitome. We then can calculate scaling factor S by Equation 3. A comprehensive analysis validating the proposed techniques can be found in ablation study.

4.3 Architecture and Data Path of EPIM

The epitome method differs from traditional convolutions in that it breaks down the convolution into multiple smaller kernels. These kernels interact with specific sections of the input feature map to produce corresponding segments of the output feature map. To navigate this during EPIM computation, we need to pinpoint the relationships between the input data and the kernel storage addresses. To accommodate this, we introduce slight changes to the data-path of current PIM accelerators, integrating three index tables as visualized in Figure 2(b).

In practice, this means selecting the right inputs from the buffer and positioning them on the appropriate word lines of PIM crossbars. Also, voltages for those word lines connected to weights not part of this computation segment should be set to zero. Similarly, the outputs from the bit line must fit correctly within the output feature map. To efficiently manage this without adding to runtime, we've enhanced the base PIM accelerators with three index tables: the Input Feature Address Table (IFAT) and Input Feature Row Table (IFRT) for the input features, and the Output Feature Address Table (OFAT) for the outputs, as shown in Figure 2(b). The remaining PIM accelerator components remain consistent with existing work [12].

The IFAT comprises a sequence of start and stop index pairs. These indices pinpoint the position of the input feature that corresponds to the epitome kernel used in the current round, and the inputs stored between these indices are necessary for computation. The quantity of these index pairs equates to the number of times the memory device crossbars are activated. The IFRT contains multiple sequences used to identify the positions of word lines where the weights connected to this word line are used in the current round. The length of the sequence is equivalent to the number of PIM crossbar rows. The quantity of sequences should be equal to the number of small patches sampled from an epitome, given that we need to use it each time we load inputs. In the EPIM data-path, the process begins with the address controller generating a continuous address. After being processed by IFAT and IFRT, the input features are correctly transferred from the buffer into the input register.

Similar to the IFAT, the OFAT also consists of start and stop index pairs. These pairs indicate the position of the result within the entire output feature. The number of these index pairs corresponds to the number of small patches. After all small patches have been activated, and before the output features are forwarded to the next layer, the OFAT comes into play. It employs the joint module to reconstruct the output feature map. This is achieved by adding the output features with identical start and stop indices and concatenating those with sequential indices.

5 EPIM DESIGN OPTIMIZATION

5.1 Motivation

We evaluated the computational overhead via simulation. Figure 4 indicates that epitomes can heighten both latency and energy consumption. In certain configurations, such as the "256x256 Epitome",

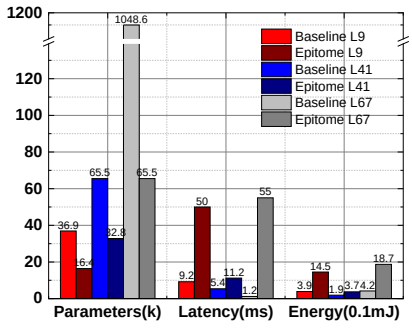


Figure 3: The parameter size, latency, and energy consumption correspond to different layers (Layer 9, 41, 67) in ResNet50, with or without epitome.

the increase is significant, with latency and energy rising to 3.86× and 2.13× that of the ResNet-50 Baseline, respectively.

Upon deeper analysis, we determined that the causes of increased latency and energy consumption differ. For latency, due to the repeated activation of memristor crossbars by the epitome, the latency of all hardware components increases proportionally. For example, if we use a epitome with 256 output channels to represent a convolution with 512 output channels, we would need to activate the memristor crossbar at least 2 times. This results in the latency of the epitome increasing proportionally with the number of times the memristor crossbars are activated. As shown in Figure 4(a), the overall latency increase is roughly proportional to the compression rate.

The increase in energy consumption, on the other hand, is a bit more complex. In essence, this increase arises due to the epitome's extensive use of certain high-cost hardware components. Still using the previous example, for this operator, since we need to store a feature map in the buffer each time we activate a small kernel, the output buffer has to be written four times more than for a convolution. As depicted by Figure 4b, the energy consumption also increases sharply with the compression rate.

To counteract this increase, we propose layer-wise epitome design and output channel wrapping.

5.2 Layer-Wise Epitome Design

Within epitome based neural networks, the sensitivity to these increases differs significantly between layers.Figure 3 portrays the parameter size, latency, and energy for three layers in ResNet-50. The darker color represents the results using epitomes, while the lighter color stands for the results of convolutions. For instance, layer 67 (depicted in grey) shows that the epitome reduces parameter size by 983.6k, while only introducing a 53.8 ms increase in latency and a 14.5 mJ increase in energy consumption. In contrast, for layer 9 (depicted in red), the epitome reduces the parameter size by merely 20.5k but introduces a 40.8 ms increase in latency and a 10.6 mJ increase in energy consumption.

Considering the varied sensitivities across layers, we propose a tailored epitome design specific to each layer. By creating smaller epitomes for layers that are less impacted by increases, and larger

Algorithm 1 Pseudo code of Evolution Search-based Epitome Design

Require: Population: $\{P\}_i$; Population Filtered by Model Size $\{O\}_i$; Evaluation Tools: ET; Evaluation results of candidates: $\{R\}_i$

1: Initialize: $\{P\}_0$.init(), i = 0
2: **for** $i < Max\ Iteration$ **do**
3: // Filter population
4: **for** each $individual$ in $\{P\}_i$ **do**
5: **if** Model Size($individual$) < Target Size **then**
6: $\{O\}_i$.append($individual$)
7: $\{R\}_i$.append(ET($individual$))
8: //Select good candidates:
9: $Parents$ = sort($\{Reward\}_i$, $\{O\}_i$)
10: // Mutate parents
11: **for** each $parent$ in $Parents$ **do**
12: $child$ = mutate($parent$, $\{P_x\}$)
13: $\{P\}_{i+1}$.append($parent$)
14: $\{P\}_{i+1}$.append($child$)
15: **return** $\{O\}_{Max\ Iteration}$

epitomes for those more sensitive, we can substantially lower both latency and energy use while keeping the same compression rate. This process can be written as an optimization problem under constraint as follows.

Let E be a set where its element e_i represents the epitome choice for the i^{th} layer in a total l layer neural network, and e_i is chosen from a set of candidate epitomes $C = \{c_1, c_2, \ldots, c_N\}$. We aim to find the optimal E^* that minimizes the total latency and energy consumption, while using less memristor crossbars as desired level.

However, deciding the optimal E^* poses a significant challenge. Given multiple epitome candidates for each layer, the entire design space boasts N^l potential combinations. This search space can be overwhelmingly vast. In our study, this search space encompasses 20,676,608 epitome combinations. To overcome this challenge, we propose to use evolution search to explore the search space.

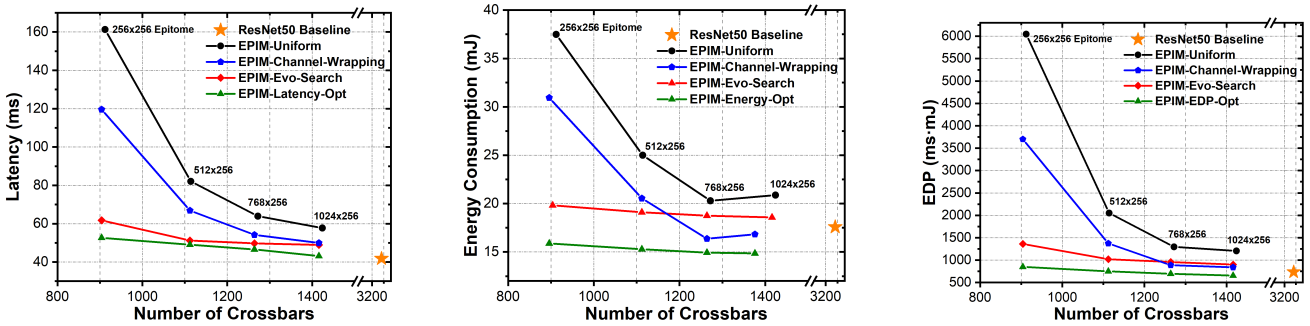
Algorithm 1 shows the details of over evolution search algorithm for deciding the optimal epitomes. We use the inverse of latency or energy as reward that the evolution search tries to optimize. To guarantee that the searched epitomes maintains a desired compression rate, we introduce a value m to consider the crossbars used by epitome. We formulate the reward in evolution search as Equation 6.

$$Reward = \frac{1}{\text{Latency}(E)} \times m \quad \text{or} \quad \frac{1}{\text{Energy}(E)} \times m, \tag{6}$$

$$\text{where} \quad m = \begin{cases} 0, & \text{if \#Crossbar}(E) > \text{Budget}, \\ 1, & \text{if \#Crossbar}(E) \leq \text{Budget}. \end{cases} \tag{7}$$

As is shown in Equation 6, if the epitomes E uses more crossbars than desired, the value m will be set as zero.Making the reward lower than any epitome that meets the desire.

In each iteration, the evolution search algorithm selects the epitome combination Es with the highest rewards as the parent combinations Ps of the next iteration (**line** 9). Then, we randomly choose some layers out of the parent combination and randomly set them to other epitomes. The above processes are repeated for many iterations. We use the best epitome combination in the last iteration as E^*.



(a) Latency (b) Energy (c) EDP(Energy-Delay Product)

Figure 4: Comparison of the uniform-epitome solution with two optimization methods: EPIM-Channel Wrapping and EPIM-Evo-Search, each used individually. EPIM-Opt refers to a combination of these two methods.

5.3 Output Channel Wrapping

We can further improve epitome performance by utilizing computational redundancy in the output channel. In our approach, the small patches sampled from epitome are replicated multiple times across output channels. If a patch has c output channels and is used to represent a convolution with $c \times r$ channels, it's duplicated r times. Representing the weight of this virtual convolution as W, we can describe the translation invariance property as:

$$W[x, :, :, :] = W[x + c, :, :, :], \quad x = 0, \ldots, c \times (r - 1) \tag{8}$$

Such replication prompts a translation invariance in the output feature map as well, which we represent as OFM. This characteristic can be formally depicted as:

$$OFM[x, :, :, :] = OFM[x + c, :, :, :], \quad x = 0, \ldots, c \times (r - 1) \tag{9}$$

With this property, we can simply calculate c channels of OFM and reuse the result for the rest. In this way, the output buffer write times is reduced by a factor of r. We refer to this method as "output channel wrapping". To support output channel wrapping, we only need to slightly modify IFAT and OFAT by changing the start and stop index of the feature maps. Together with optimization in algorithms, the performance of EPIM will be further improved. An evaluation of the two optimizations is presented in Figure 4.

6 EXPERIMENTS

6.1 Experimental Setup

The development of the simulator is based on MNSIM [13], a highly accurate, behavior-level simulator. Following the practice in MNSIM, we maintain a look-up table for the storage of the latency and power parameters associated with basic hardware behaviors. The lookup table is modified to include the parameters of epitome. We use the well-explored 2-bit memristor cells in simulation. For quantization, we integrate the popular HAWQ [5] algorithm into our simulator. All the experiments are conducted on the large-scale benchmark ImageNet [3]. For comparison purposes, we reproduced PIM-Prune, a pruning framework designed for neural networks on PIM accelerators[2].

6.2 Results and Analyses

The main experiment results of EPIM are summarized in Table 1 and Figure 4.

Table 1 provides a comparison between conventional NN algorithms and our EPIM solutions. We choose ResNet-50 and ResNet-101 to demonstrate the experimental results. For ResNet50, our method achieves up to a 30.65× compression rate of crossbars with a maximum accuracy loss of 4.78%. For ResNet-101, our method achieves up to a 31.22× compression rate of crossbars with a maximum accuracy loss of 3.79%. More importantly, combined with ultra-low bit quantization, EPIM reduces the inference energy by 23.01 × for ResNet-50 and 22.64× for ResNet-101. Our experiments show that EPIM can serve as a competitive solution to achieve efficient PIM deployment of large CNNs.

In Figure 4, we contrast manually designed uniform epitomes with epitomes produced by design optimization methods. The EPIM-Opt method, combining Channel Wrapping and Evo-Search, outperforms others. Compared to uniform epitomes, it offers similar compression with up to 3.07× speedup, 2.36× energy savings, and 7.13× reduced EDP. A comparison of ResNet50 under W9A9 quantization in Table 1 reveals that EPIM-ResNet50-Latency-Opt exhibits the lowest latency, while EPIM-ResNet50-Energy-Opt presents the lowest energy consumption, both without significant accuracy degradation.

7 ABLATION STUDY

7.1 Detailed Experiments for Quantization

The detailed results of quantization are provided in Table 2. We retrained six quantized models under two ultra-low-bit quantization scenarios. The results show that both of the proposed updates contribute to the increase in accuracy. For instance, for ResNet-50 under 3-bit quantization, the accuracy can be improved from 69.95% to 71.35% by adjusting scaling factor with crossbars. Moreover, by using the weighted sum scaling factor, the accuracy can be further improved to 71.59%.

7.2 Detailed Comparison of Epitome and Pruning

In order to assess the epitome's compatibility with pruning, we experiment with merging both concepts using basic element-wise

Table 1: Experimental results of EPIM on ImageNet. CR stands for compression rate, XB stands for crossbar array, and mp stands for mixed-precision. The dimensions of an epitome are represented by a product notation. For example, "1024×256" represents that $c_{in} \times p \times q = 1024$ and $c_{out} = 256$

Model	Bitwidth	Epitome	Accuracy(%) ↑	# XBs ↓	CR of XBs ↑	Latency(ms) ↓	Energy(mJ) ↓	Memristor Utilization(%) ↑
ResNet50	FP32	-	76.37	13120	1.00	139.8	214.0	94.9
EPIM-ResNet50	FP32	1024×256	74.00	5696	2.30	167.7	194.8	96.7
PIM-Prune-ResNet50[2]	FP32	-	73.18	-	2.18	-	-	-
EPIM-ResNet50	W9A9	1024×256	73.98	1424	9.21	50.9	17.0	96.7
EPIM-ResNet50-Latency-Opt	W9A9	layer-wise	73.60	1080	12.15	49.2	16.4	93.4
EPIM-ResNet50-Energy-Opt	W9A9	layer-wise	73.15	1048	12.52	50.6	15.6	93.2
EPIM-ResNet50	W7A9	1024×256	73.81	1076	12.19	45.2	20.5	93.2
EPIM-ResNet50	W5A9	1024×256	73.59	720	18.12	39.9	13.7	93.2
EPIM-ResNet50	W3mpA9	1024×256	72.98	618	21.23	37.0	10.2	93.2
EPIM-ResNet50	W3A9	1024×256	71.59	428	30.65	36.7	9.3	93.2
ResNet101	FP32	-	78.77	22912	1.00	189.7	385.7	94.7
EPIM-ResNet101	FP32	1024×256	76.56	10592	2.16	263.7	364.8	98.2
PIM-Prune-ResNet101[2]	FP32	-	75.82	-	1.90	-	-	-
EPIM-ResNet101	W9A9	1024×256	76.52	2648	8.65	75.8	32.2	98.2
EPIM-ResNet101	W7A9	1024×256	76.48	1994	11.49	73.7	39.5	98.2
EPIM-ResNet101	W5A9	1024×256	75.68	1584	14.46	72.1	29.2	98.2
EPIM-ResNet101	W3mpA9	1024×256	75.80	1052	21.78	65.5	18.6	98.2
EPIM-ResNet101	W3A9	1024×256	74.98	734	31.22	63.4	17.0	98.2

Table 2: Detailed Quantization Experiments Results.

Model	Accuracy (%)		
	Naïve Quant	+ Adjust with Crossbars	+ Adjusted with Overlap
ResNet-50 (3-bit)	69.95	71.35	71.59
ResNet-50 (3-5 bit)	72.18	72.83	72.98
ResNet-101 (3-bit)	73.98	74.96	74.98
ResNet-101 (3-5 bit)	75.46	75.71	75.80

pruning methods. Due to challenges in determining the crossbar compression rate with pruning, we compare parameter compression rates. Results are in Table 3.

Table 3: Accuracy and compression rate of epitome, epitome + element pruning with 50% pruning ratio, PIM-Prune[2].

Method	ResNet-50		ResNet-101	
	Accuracy (%)	Compress. Rate	Accuracy (%)	Compress. Rate
Epitome	74.00	2.25	76.56	2.08
Epitome + Pruning	73.18	3.49	75.76	3.64
PIM-Prune 50%	72.77	1.80	75.82	1.90
PIM-Prune 75%	72.19	3.38	74.80	3.24

8 CONCLUSION

In this paper, we introduce the epitome for model compression in PIM accelerators. We redesign the PIM architecture and utilize epitome-aware quantization, layer-wise epitome design and output channel wrapping to enhance performance. Our EPIM method achieves a 71.59% top-1 accuracy on ImageNet using 3-bit EPIM-ResNet50, reducing crossbar areas by 30.65×.

REFERENCES

[1] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 27–39.

[2] Chaoqun Chu, Yanzhi Wang, Yilong Zhao, Xiaolong Ma, Shaokai Ye, Yunyan Hong, Xiaoyao Liang, Yinhe Han, and Li Jiang. 2020. Pim-prune: Fine-grain dcnn pruning for crossbar-based process-in-memory architecture. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

[3] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 248–255.

[4] Zhen Dong, Zhewei Yao, Daiyaan Arfeen, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. 2020. HAWQ-V2: Hessian Aware trace-Weighted Quantization of Neural Networks. *Advances in neural information processing systems* (2020).

[5] Zhen Dong, Zhewei Yao, Amir Gholami, Michael W Mahoney, and Kurt Keutzer. 2019. Hawq: Hessian aware quantization of neural networks with mixed-precision. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 293–302.

[6] Weiwen Jiang, Qiuwen Lou, Zheyu Yan, Lei Yang, Jingtong Hu, Xiaobo Sharon Hu, and Yiyu Shi. 2020. Device-circuit-architecture co-exploration for computing-in-memory neural accelerators. *IEEE Trans. Comput.* 70, 4 (2020), 595–605.

[7] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 14–26.

[8] Hanbo Sun, Chenyu Wang, Zhenhua Zhu, Xuefei Ning, Guohao Dai, Huazhong Yang, and Yu Wang. 2022. Gibbon: efficient co-exploration of NN model and processing-in-memory architecture. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 867–872.

[9] Hanbo Sun, Zhenhua Zhu, Yi Cai, Xiaoming Chen, Yu Wang, and Huazhong Yang. 2020. An energy-efficient quantized and regularized training framework for processing-in-memory accelerators. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 325–330.

[10] Weier Wan, Rajkumar Kubendran, Clemens Schaefer, Sukru Burc Eryilmaz, Wenqiang Zhang, Dabin Wu, Stephen Deiss, Priyanka Raina, He Qian, Bin Gao, et al. 2022. A compute-in-memory chip based on resistive random-access memory. *Nature* 608, 7923 (2022), 504–512.

[11] Daquan Zhou, Xiaojie Jin, Qibin Hou, Kaixin Wang, Jianchao Yang, and Jiashi Feng. 2019. Neural epitome search for architecture-agnostic network compression. *arXiv preprint arXiv:1907.05642* (2019).

[12] Zhenhua Zhu, Hanbo Sun, Yujun Lin, Guohao Dai, Lixue Xia, Song Han, Yu Wang, and Huazhong Yang. 2019. A configurable multi-precision CNN computing framework based on single bit RRAM. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.

[13] Zhenhua Zhu, Hanbo Sun, Kaizhong Qiu, Lixue Xia, Gokul Krishnan, Guohao Dai, Dimin Niu, Xiaoming Chen, X Sharon Hu, Yu Cao, et al. 2020. MNSIM 2.0: A behavior-level modeling tool for memristor-based neuromorphic computing systems. In *Proceedings of the 2020 on Great Lakes Symposium on VLSI*. 83–88.